

How to use the Pandas set_index() and reset_index() functions

Learn how to use the Pandas set_index() and reset_index() functions to add and remove a single or multi-level index to your dataframe.



📷 Picture by Nothing Ahead, Pexels.
🕒 11 minutes to read

Data Science Pandas

While many Pandas operations don’t require or benefit from an explicitly named index on the dataframe, named indexes (or indices) can be beneficial for some tasks because a wide range of functions are designed to let you manipulate the data based on index values.

By default, Pandas will add a basic numeric index and this is fine for most tasks. However, you can also add explicitly named indexes with a single level or a multi-level or hierarchical index using the `pandas.DataFrame.set_index()` method. Similarly, you can remove, drop, or reset indexes using the `pandas.DataFrame.reset_index()` method. In this quick tutorial I’ll show you how to use them.

Create a dataframe with a single index

To get started, open a Jupyter notebook and import the Pandas library using the `import pandas as pd` naming convention. Then either import data into Pandas from a CSV file or create a dataframe of dummy data to work with. Note that when the dataframe is created, Pandas will add its own numeric index without the need for you to explicitly set a column to use as the index.

```
import pandas as pd
```

```
df = pd.DataFrame([
    ('Pterophyllum altum', 'Pterophyllum', 12.5),
    ('Coptodon snyderae', 'Coptodon', 8.2),
    ('Coptodon discolor', 'Coptodon', 18.5),
    ('Astronotus ocellatus', 'Astronotus', 31.2),
    ('Astronotus crassipinnis', 'Astronotus', 31.2),
    ('Corydoras aeneus', 'Corydoras', 5.3),
    ('Corydoras paleatus', 'Corydoras', 6.3),
    ('Xenomystus nigri', 'Xenomystus', 5.3)
],
    columns=['species', 'genus', 'length_cm']
)

df
```

	species	genus	length_cm
0	Pterophyllum altum	Pterophyllum	12.5
1	Coptodon snyderae	Coptodon	8.2
2	Coptodon discolor	Coptodon	18.5
3	Astronotus ocellatus	Astronotus	31.2
4	Astronotus crassipinnis	Astronotus	31.2
5	Corydoras aeneus	Corydoras	5.3
6	Corydoras paleatus	Corydoras	6.3
7	Xenomystus nigri	Xenomystus	5.3

Use set_index() to add a single-level index

If you wish to designate a specific column in your dataframe to become the index you can do so using the Pandas `pandas.DataFrame.set_index()` function. For a basic single-level dataframe this takes a string value corresponding to the name of the column you wish to set as the index.

The `set_index()` is called using dot notation to append it to the dataframe object and returns a dataframe without transforming the original. If you want to save the output and overwrite the original dataframe object you need to redeclare it, i.e. `df = df.set_index('species')`.

The function does include an `inplace=True` option to avoid the need to do this, but `inplace=True` is set to be removed from Pandas in the future, so it’s best to avoid using it in your code. After running the code you’ll see that the numeric index has been replaced using the values from the `species` column.

```
df = df.set_index('species')

df
```

	genus	length_cm
species		
Pterophyllum altum	Pterophyllum	12.5
Coptodon snyderae	Coptodon	8.2

	genus	length_cm
species		
Coptodon discolor	Coptodon	18.5
Astronotus ocellatus	Astronotus	31.2
Astronotus crassipinnis	Astronotus	31.2
Corydoras aeneus	Corydoras	5.3
Corydoras paleatus	Corydoras	6.3
Xenomystus nigri	Xenomystus	5.3

Use reset_index() to reset the index

To remove the index and revert to a regular numeric index you can use the `pandas.DataFrame.reset_index()` method. This can accept a few different parameters but the defaults are usually sufficient in most cases.

Calling `df = df.reset_index()` will remove the `species` column currently being used for the index and revert to use it as a regular column or series in the dataframe by replacing it with a numeric index.

As with `set_index()`, the `reset_index()` function takes a dataframe as its input and returns a dataframe as its output, so must either be used with its optional `inplace=True` parameter, or better used to overwrite the original dataframe object.

```
df = df.reset_index()
df
```

	species	genus	length_cm
0	Pterophyllum altum	Pterophyllum	12.5
1	Coptodon snyderae	Coptodon	8.2
2	Coptodon discolor	Coptodon	18.5
3	Astronotus ocellatus	Astronotus	31.2
4	Astronotus crassipinnis	Astronotus	31.2
5	Corydoras aeneus	Corydoras	5.3
6	Corydoras paleatus	Corydoras	6.3
7	Xenomystus nigri	Xenomystus	5.3

Use reset_index() to reset and remove the index

Sometimes, your index might be so useless that you want to both reset it and revert to a numeric index and also remove it entirely from your dataframe. You can do this using the `drop=True` argument.

First, we'll set the species column to be the index of the dataframe, then we'll call `df = df.reset_index('species', drop=True)` to reset the index and remove or drop the species column afterwards. This returns a new dataframe that we'll use to overwrite the original, which now contains only two columns, genus and length_cm.

```
df = df.set_index('species')
df
```

	genus	length_cm
species		
Pterophyllum altum	Pterophyllum	12.5
Coptodon snyderae	Coptodon	8.2
Coptodon discolor	Coptodon	18.5
Astronotus ocellatus	Astronotus	31.2
Astronotus crassipinnis	Astronotus	31.2
Corydoras aeneus	Corydoras	5.3
Corydoras paleatus	Corydoras	6.3
Xenomystus nigri	Xenomystus	5.3

```
df = df.reset_index('species', drop=True)
df
```

	genus	length_cm
0	Pterophyllum	12.5
1	Coptodon	8.2
2	Coptodon	18.5
3	Astronotus	31.2
4	Astronotus	31.2
5	Corydoras	5.3
6	Corydoras	6.3
7	Xenomystus	5.3

Create a dataframe with a multi-level index

Pandas also supports the creation of dataframes with multi-level indexes. They can be beneficial for displaying certain datasets, but I tend to only use them relatively rarely myself.

Multi-level indexes (or indices) can be added to Pandas dataframes by passing a list of column names to the `set_index()` function. For example, if we pass `['species', 'genus']` to `set_index()` Pandas will add an index with two columns.


```
df = pd.DataFrame([
    ('Pterophyllum altum', 'Pterophyllum', 12.5),
    ('Coptodon snyderae', 'Coptodon', 8.2),
    ('Coptodon discolor', 'Coptodon', 18.5),
    ('Astronotus ocellatus', 'Astronotus', 31.2),
    ('Astronotus crassipinnis', 'Astronotus', 31.2),
    ('Corydoras aeneus', 'Corydoras', 5.3),
    ('Corydoras paleatus', 'Corydoras', 6.3),
    ('Xenomystus nigri', 'Xenomystus', 5.3)
],
    columns=['species', 'genus', 'length_cm']
).set_index(['species', 'genus'])

df
```

		length_cm
species	genus	
Pterophyllum altum	Pterophyllum	12.5
Coptodon snyderae	Coptodon	8.2
Coptodon discolor	Coptodon	18.5
Astronotus ocellatus	Astronotus	31.2
Astronotus crassipinnis	Astronotus	31.2
Corydoras aeneus	Corydoras	5.3
Corydoras paleatus	Corydoras	6.3
Xenomystus nigri	Xenomystus	5.3

Use reset_index() to reset both index levels

If you want to remove a multi-level index from a Pandas dataframe you can use the `reset_index()` method we covered above. By default, if you don't define which index level you wish to remove, Pandas will remove them all and the index values will revert to being columns within the dataframe.

```
df.reset_index()
```

	species	genus	length_cm
0	Pterophyllum altum	Pterophyllum	12.5
1	Coptodon snyderae	Coptodon	8.2

	species	genus	length_cm
2	Coptodon discolor	Coptodon	18.5
3	Astronotus ocellatus	Astronotus	31.2
4	Astronotus crassipinnis	Astronotus	31.2
5	Corydoras aeneus	Corydoras	5.3
6	Corydoras paleatus	Corydoras	6.3
7	Xenomystus nigri	Xenomystus	5.3

Use reset_index() to reset a single index level

If you want to modify a dataframe with a multi-level index to become one with a single-level index you can explicitly name the index level you wish to reset. The numbers used go from zero upwards, so to reset the genus index and revert its values to those of a regular column or series we'd enter `df.reset_index(level=1)`, because it's the second index level.

```
df
```

		length_cm
species	genus	
Pterophyllum altum	Pterophyllum	12.5
Coptodon snyderae	Coptodon	8.2
Coptodon discolor	Coptodon	18.5
Astronotus ocellatus	Astronotus	31.2
Astronotus crassipinnis	Astronotus	31.2
Corydoras aeneus	Corydoras	5.3
Corydoras paleatus	Corydoras	6.3
Xenomystus nigri	Xenomystus	5.3

```
df.reset_index(level=1)
```

	genus	length_cm
species		
Pterophyllum altum	Pterophyllum	12.5
Coptodon snyderae	Coptodon	8.2
Coptodon discolor	Coptodon	18.5
Astronotus ocellatus	Astronotus	31.2
Astronotus crassipinnis	Astronotus	31.2

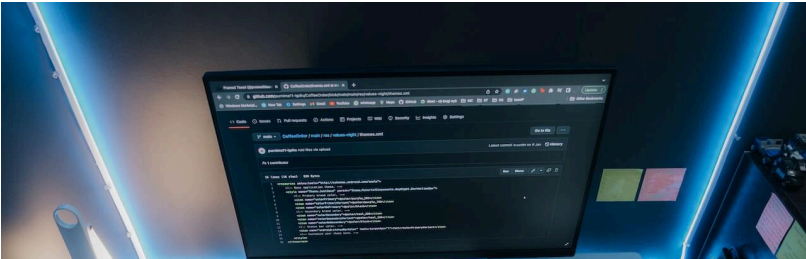
	genus	length_cm
species		
Corydoras aeneus	Corydoras	5.3
Corydoras paleatus	Corydoras	6.3
Xenomystus nigri	Xenomystus	5.3

Matt Clarke, Sunday, January 01, 2023



Matt Clarke

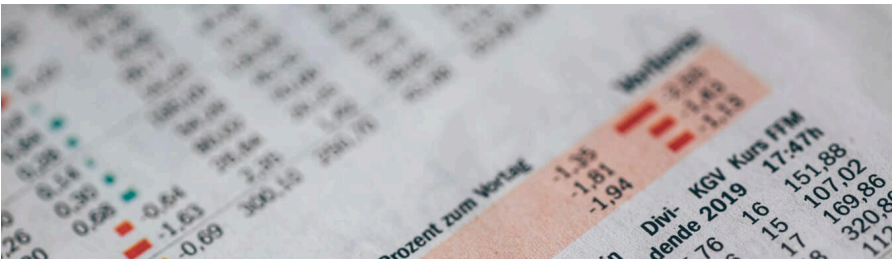
Matt is an Ecommerce and Marketing Director who uses data science to help in his work. Matt has a Master's degree in Internet Retailing (plus two other Master's degrees in different fields) and specialises in the technical side of ecommerce and marketing.



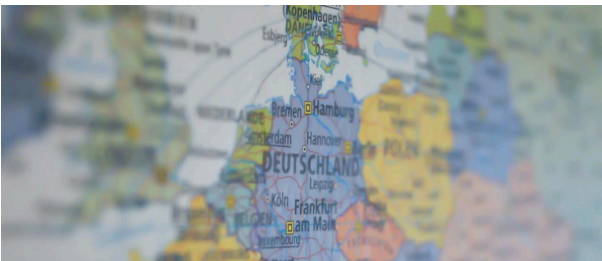
How to use sort_values() to sort a Pandas DataFrame



How to transpose a Pandas dataframe using T and transpose()



How to select, filter, and subset data in Pandas dataframes



How to use the Pandas map() function



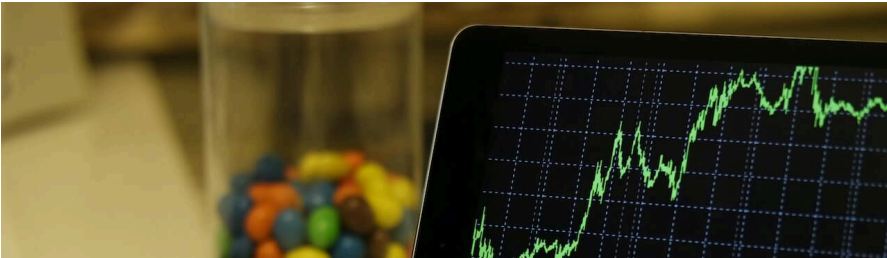
How to use Category Encoders to encode categorical variables



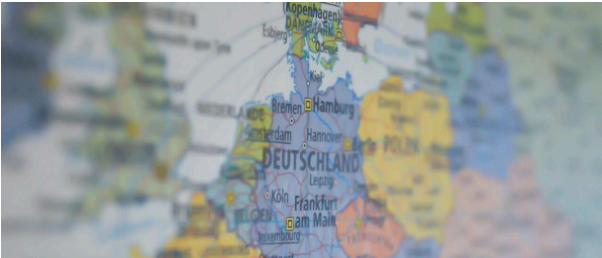
How to identify and remove duplicate values in Pandas



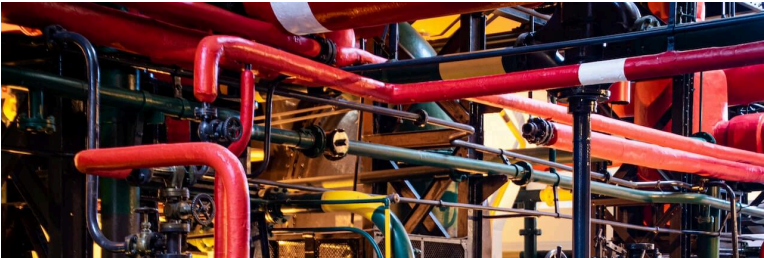
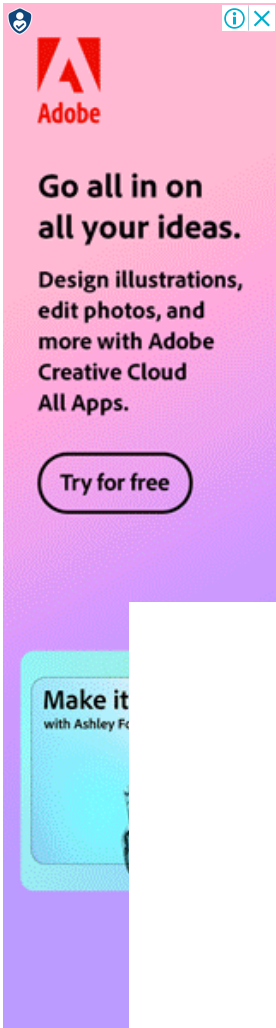
How to use Pandas from_records() to create a dataframe



How to calculate an exponential moving average in Pandas



How to use the Pandas map() function



How to use Pandas pipe() to create data pipelines



How to use Pandas assign() to create new dataframe columns

How to measure Python code execution times with timeit



How to use Pandas from_records() to create a dataframe

How to calculate an exponential moving average in Pandas

How to use the Pandas map() function

How to use Pandas `pipe()` to create data pipelines

How to use Pandas `assign()` to create new dataframe columns

How to measure Python code execution times with `timeit`

Other posts you might like

How to use the Pandas `truncate()` function

Have you ever needed to chop the top or bottom off a Pandas dataframe, or extract a specific section from the middle? If so, there’s a Pandas function called `truncate()`...

How to use Spacy for noun phrase extraction

Noun phrase extraction is a Natural Language Processing technique that can be used to identify and extract noun phrases from text. Noun phrases are phrases that function grammatically as nouns...

How to use the Pandas filter() function

The Pandas filter() function is used to filter a dataframe based on the column names, rather than the column values, and is useful in creating a subset dataframe containing only...

Get the newsletter

Subscribe to receive data science updates

Enter your email

Subscribe

Categories

- [Data Science](#)
- [Machine Learning](#)
- [Data Engineering](#)

Site

- [Sitemap](#)
- [About](#)
- [LinkedIn](#)
- [GitHub](#)

The Practical Data Science blog

The Practical Data Science blog is written by [Matt Clarke](#), an Ecommerce and Marketing Director who specialises in data science and machine learning for marketing and retail. The site provides articles and tutorials on data science, machine learning, and data engineering to help you improve your business and your data science skills.

Copyright © 2023 Practical Data Science